Creating PivotTable Reports and Charts with VBA in Excel 2010

Office 2010

46 out of 62 rated this helpful - Rate this topic

Summary: Learn how to use Visual Basic for Applications (VBA) in Microsoft Excel 2010 to create and manipulate PivotTable reports and charts. Using code to create PivotTable reports and charts can help you to do these tasks repeatedly and more efficiently.

Last modified: June 05, 2011

Applies to: *Excel 2010 | Office 2010 | SharePoint Server 2010 | VBA*

In this article

Overview of Using VBA with PivotTable Reports and Charts
Creating a PivotTable Report by Using VBA
Creating a Chart by Using VBA
Making Changes to an Embedded Chart
Creating a Chart Based on a PivotTable Report
Conclusion
Additional Resources

Published: June 2011

Provided by: Dr. Gerard M. Verschuuren, Mr. Excel | Mark Roberts, Microsoft Corporation

Contents

- Overview of Using VBA with PivotTable Reports and Charts
- Creating a PivotTable Report by Using VBA
- Creating a Chart by Using VBA
- Making Changes to an Embedded Chart
- Creating a Chart Based on a PivotTable Report
- Conclusion
- Additional Resources

Download sample workbook: Creating PivotTable Reports and Charts with VBA (PivotTablesAndCharts.xlsm)

Overview of Using VBA with PivotTable Reports and Charts

By reading this article and downloading the sample workbook, you can learn how to create PivotTable reports and charts by using Visual Basic for Applications (VBA) code. To run the code described in this article with sample data, download the PivotTablesAndCharts sample workbook.

Creating a PivotTable Report by Using VBA

The PivotTable report created by the CreatePivot macro in the PivotTablesAndCharts sample workbook is shown in Figure 1.

Figure 1. PivotTable report

	A	B	C	D	E	F
1	GENDER	(All)				
2						
3	Sum of SALARY	LOCATION				
4	DEPT	Boston	Cambridge	Marlboro	Worcester	Grand Total
5	Accounting		$ 41,745	$ 54,757		$ 96,502
6	Communication	$ 109,808				$ 109,808
7	Finance	$ 40,746	$ 54,450			$ 95,196
8	Management	$ 87,692			$ 186,014	$ 273,706
9	Marketing	$ 80,526		$ 53,240		$ 133,766
10	Planning	$ 37,400	$ 33,674	$ 88,578	$ 106,879	$ 266,532
11	Sales	$ 41,250		$ 77,949		$ 119,199
12	Grand Total	$ 397,421	$ 129,869	$ 274,524	$ 292,893	$ 1,094,708

This PivotTable report is based on the table of data on the Employees worksheet of the sample workbook. As shown in the following lines of code, the CreatePivot macro activates the worksheet and then uses the **PivotTableWizard** method to start the process of creating the PivotTable report.

VBA

```
ActiveWorkbook.Sheets("Employees").Select
Range("A1").Select
Set objTable = Sheet1.PivotTableWizard
```

Note

If you press F8 in the code editor to step through the CreatePivot macro code line by line, you can see that the li code that calls the**PivotTableWizard** method adds a new worksheet to the workbook and then creates the new PivotTable report on that sheet.

The **PivotTableWizard** method returns a **PivotTable** object, which has a **PivotFields** collection associated with it. The CreatePivot macro continues by adding**PivotField** objects to the **PivotFields** collection to specify the following PivotTable report fields.

- A row field named DEPT
- A column field named LOCATION
- A data field based on the SALARY field that uses the SUM function

VBA

```
Set objField = objTable.PivotFields("DEPT")
objField.Orientation = xlRowField

Set objField = objTable.PivotFields("LOCATION")
objField.Orientation = xlColumnField

Set objField = objTable.PivotFields("SALARY")
objField.Orientation = xlDataField
objField.Function = xlSum
objField.NumberFormat = "$ #,##0"
```

Note

Although this example uses only one row field and one column field, you can add additional fields to the **PivotFields** collection to create PivotTable reports that have multiple levels in each dimension.

The CreatePivot macro also adds a page field to enable filtering the report by the GENDER field, as shown in the following lines of code.

VBA
```
Set objField = objTable.PivotFields("GENDER")
objField.Orientation = xlPageField
```

The CreatePivot macro then shows the new PivotTable report in print preview.

VBA
```
ActiveSheet.PrintPreview
```

As shown in the following lines of code, after the user closes print preview, the CreatePivot macro asks the user whether to delete the new PivotTable report. The line that sets the **DisplayAlerts** property to **False** suppresses the built-in warning from Microsoft Excel that "Data may exist in the sheet(s) selected for deletion."

VBA
```
Application.DisplayAlerts = False
If MsgBox("Delete the PivotTable?", vbYesNo) = vbYes Then
    ActiveSheet.Delete
End If
Application.DisplayAlerts = True
```

The following listing shows the entire code for the CreatePivot macro.

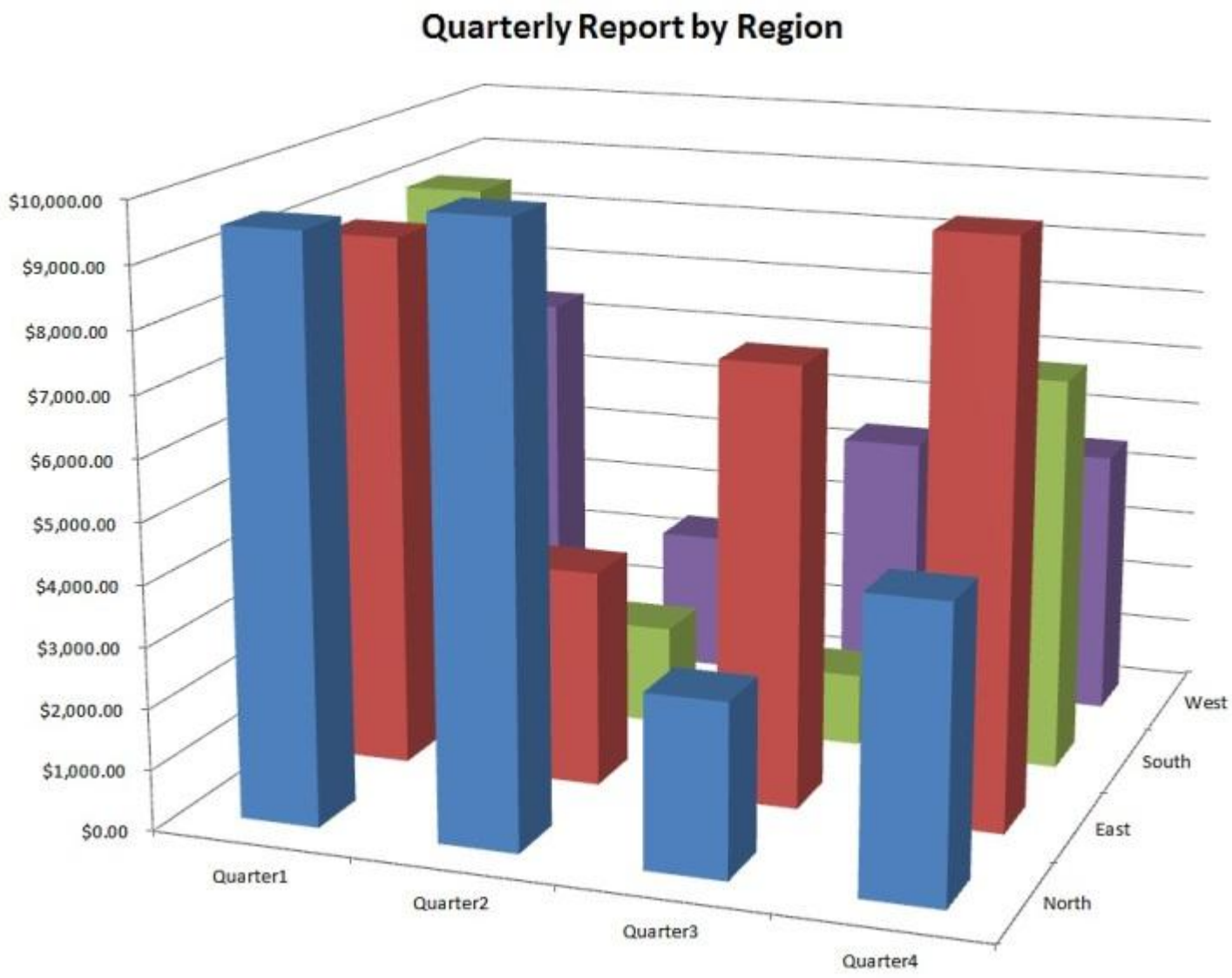
VBA
```
Sub CreatePivot()
    ' Creates a PivotTable report from the table on Sheet1
    ' by using the PivotTableWizard method with the PivotFields
    ' method to specify the fields in the PivotTable.
    Dim objTable As PivotTable, objField As PivotField

    ' Select the sheet and first cell of the table that contains the data.
    ActiveWorkbook.Sheets("Employees").Select
    Range("A1").Select

    ' Create the PivotTable object based on the Employee data on Sheet1.
    Set objTable = Sheet1.PivotTableWizard

    ' Specify row and column fields.
    Set objField = objTable.PivotFields("DEPT")
    objField.Orientation = xlRowField
    Set objField = objTable.PivotFields("LOCATION")
    objField.Orientation = xlColumnField

    ' Specify a data field with its summary
    ' function and format.
    Set objField = objTable.PivotFields("SALARY")
    objField.Orientation = xlDataField
    objField.Function = xlSum
    objField.NumberFormat = "$ #,##0"

    ' Specify a page field.
    Set objField = objTable.PivotFields("GENDER")
    objField.Orientation = xlPageField

    ' Preview the new PivotTable report.
    ActiveSheet.PrintPreview
```

```
    ' Prompt the user whether to delete the PivotTable.
    Application.DisplayAlerts = False
    If MsgBox("Delete the PivotTable?", vbYesNo) = vbYes Then
        ActiveSheet.Delete
    End If
    Application.DisplayAlerts = True
End Sub
```

Creating a Chart by Using VBA

The chart created by the CreateChart macro in the sample workbook is shown in Figure 2.

Figure 2. 3-D column chart



This chart is based on the data on the Table worksheet of the sample workbook.

The initial lines of the CreateChart macro select the Table worksheet and then prompt the user to select the range of data to chart. Setting the *Default* parameter of the **InputBox** method to `Selection.Address` and setting the *Type* parameter to 8 returns a **Range** object with its **Address** property set to the range reference of the range selected by the user.

VBA

```
ActiveWorkbook.Sheets("Table").Select
```

```
Set objSelection = _
    Application.InputBox(Prompt:="Select the columns and rows to chart", _
    Default:=Selection.Address, _
    Type:=8)
```

The next lines of code verify that more than one cell was selected and, if not, prompt the user to make an appropriate selection.

VBA

```
If objSelection.Cells.Count = 1 Then
    MsgBox "You must select at least one row or column for the chart range."
    Exit Sub
End If
```

The following lines of code do the work to create the chart. The **Add** method of the **Charts** collection creates a new chart sheet and then returns a **Chart** object that is assigned to the **objChart** variable. The **With** statement then uses that variable to specify the appearance of the chart, starting by using the **SetSourceData**method to specify the range of data the user selected. Notice that the line `.Legend.Delete` is used to delete the legend because it duplicates the information displayed on the third axis of a 3-D column chart.

VBA

```
Set objChart = Charts.Add
With objChart
    .SetSourceData objSelection
    .ChartType = xl3DColumn
    .Location xlLocationAsNewSheet
    .Legend.Delete
    .PlotBy = xlColumns
End With
```

Note

This example creates a fairly simple chart. The properties and methods of the **Chart** object are quite extensive. the **Chart Members** topic to learn about the different properties and methods you can use to specify the appear of a chart.

The remaining lines of code in the CreateChart macro provide some examples of prompting the user to specify additional properties that affect the appearance of the chart, and also asking the user whether to delete the chart. The following listing shows the entire code for the CreateChart macro.

VBA

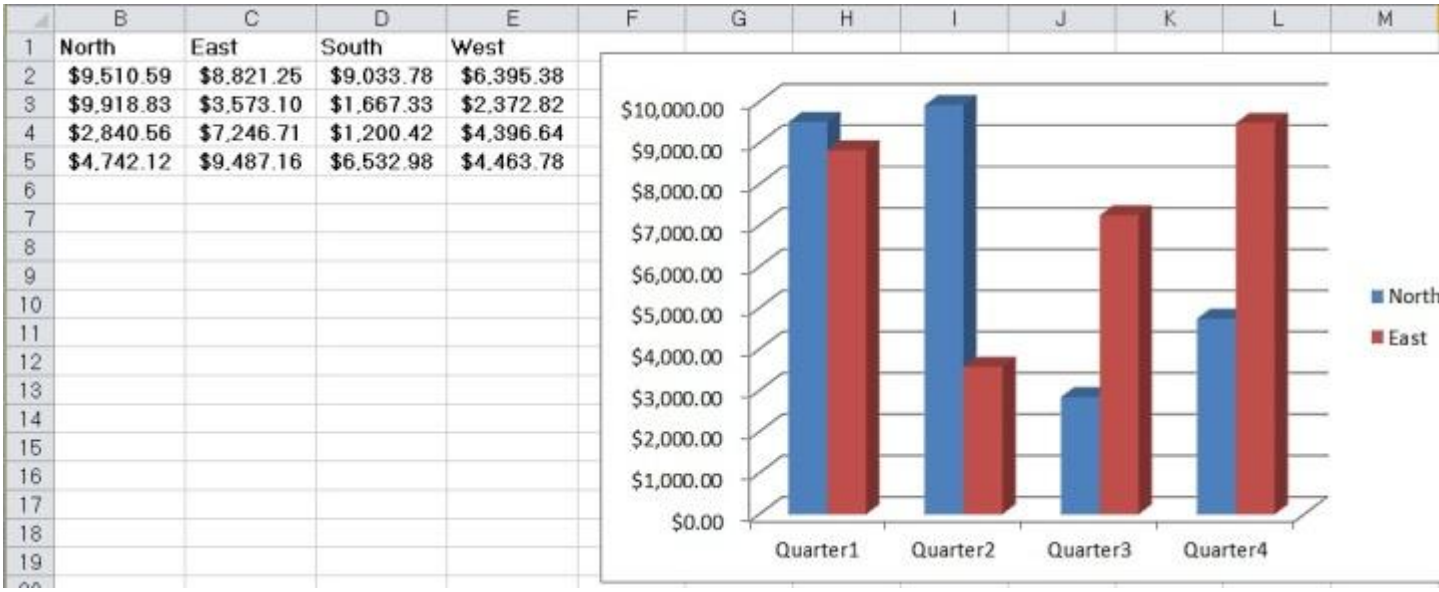
```
Sub CreateChart()
    ' Create a new chart sheet from the table on Sheet2
    ' by using the Add method of the Charts collection.
    Dim objSelection As Range, objChart As Chart

    ' Select the sheet that contains the data.
    ActiveWorkbook.Sheets("Table").Select

    ' Prompt the user to select the range to chart
    ' and set the Range object to the specified range.
    Set objSelection = _
        Application.InputBox(Prompt:="Select the columns and rows to chart",
_
        Default:=Selection.Address, _
```

```
        Type:=8)

    ' Verify whether a selection was made.
    If objSelection.Cells.Count = 1 Then
        MsgBox "You must select at least one row or column for the chart
range."
        Exit Sub
    End If

    ' Create a new chart sheet and specify its source data
    ' and appearance.
    Set objChart = Charts.Add
    With objChart
        .ChartType = xl3DColumn
        .SetSourceData objSelect
        .Location xlLocationAsNewSheet
        .Legend.Delete
        .PlotBy = xlColumns
    End With

    ' Ask the user whether to plot by rows instead.
    If MsgBox("Or plot by rows?", vbYesNo) = vbYes Then
        objChart.PlotBy = xlRows
    End If

    ' Prompt the user for a title.
    objChart.HasTitle = True
    objChart.ChartTitle.Text = InputBox("Title?")

    ' Ask the user whether to delete the chart.
    Application.DisplayAlerts = False
    If MsgBox("Delete chart?", vbYesNo) = vbYes Then
        ActiveSheet.Delete
    End If
    Application.DisplayAlerts = True
End Sub
```

Making Changes to an Embedded Chart

The DynamicChart macro in the sample workbook shows how to change the appearance of an embedded chart based on user selection of entire rows and columns. The appearance of the chart after running the macro and then selecting the North and East columns of data is shown in Figure 3.

Figure 3. Dynamic chart



To access and work with an embedded chart, you must use the ChartObjects collection of the **Worksheet** object, instead of the **Charts** collection of the**Workbook** object that is used in the example in the preceding section of this article. The initial lines of code in the DynamicChart macro show how to do this by activating the worksheet that contains the embedded chart, and then setting a **Chart** object variable to the first item in the **ChartObjects** collection of that sheet.

VBA

```
ActiveWorkbook.Sheets("Table+Chart").Activate
Set objChart = ActiveSheet.ChartObjects(1).Chart
```

The next lines of code prompt the user to select rows or columns to chart, and then assign that selection to a **Range** object variable by using the same technique shown in the Creating a Chart by Using VBA section of this article.
Based on the user's selection, the code determines whether to use the first row or first column for the categories of the chart. To do that, the number of rows selected is compared to the number of columns selected. If the number of rows is greater, the values in the first column are selected as categories. If the number of columns is greater, the values in the first row are selected as categories.

VBA

```
r = objSelection.Rows.Count
c = objSelection.Columns.Count
If r > c Then
    Set objCategories = Range(Cells(1, 1), Cells(r, 1))
Else
    Set objCategories = Range(Cells(1, 1), Cells(1, c))
End If
```

The last lines of code in the DynamicChart macro use the Union method to create a single range from the user's selection and the row or column the code determined to use for categories. Finally, the code passes that range to the **SetSourceData** method to update the display of the chart.

VBA

```
Set objSrcData = Union(objCategories, objSelection)
objChart.SetSourceData objSrcData
```

The following code sample shows the entire listing for the DynamicChart macro.

VBA

```
Sub DynamicChart()
    ' Adjusts the embedded chart on Sheet3.
    Dim objChart As Chart, objChObject As ChartObject
    Dim objSelection As Range, objSrcData As Range, objCategories As Range
```

```
    Dim r As Long, c As Long

    ' Activate the sheet that contains the chart.
    ActiveWorkbook.Sheets("Table+Chart").Activate

    ' Access the chart from the ChartObject collection
    ' of the active sheet.
    Set objChart = ActiveSheet.ChartObjects(1).Chart

    ' Prompt user to select the rows or columns to chart
    ' and set the Range object to the specified range.
    Set objSelection = _
        Application.InputBox(Prompt:="Select entire rows or columns to
chart", _
        Default:=Selection.Address, _
        Type:=8)

    ' Determine whether the user selected rows or columns,
    ' and then use either the first row or first column
    ' as the range for categories.
    r = objSelection.Rows.Count
    c = objSelection.Columns.Count
    If r > c Then
        Set objCategories = Range(Cells(1, 1), Cells(r, 1))
    Else
        Set objCategories = Range(Cells(1, 1), Cells(1, c))
    End If

    ' Create a single range from the union of
    ' categories and selected data, and then
    ' update the chart.
    Set objSrcData = Union(objCategories, objSelection)
    objChart.SetSourceData objSrcData
End Sub
```

Creating a Chart Based on a PivotTable Report

You can also create charts based on a PivotTable report as demonstrated by the CreateChartForPivot macro in the sample workbook. The macro starts this process by calling the CreatePivot macro described in the **Creating a PivotTable Report by Using VBA** section of this article.

One of the main items of interest is the line of code that accesses the newly created PivotTable report from the **PivotTables** collection of the new worksheet.

VBA
```
Set objPivot = ActiveSheet.PivotTables(1)
```

Another item of interest is the line of code that uses the **PivotTable** property to create a range that includes the entire PivotTable report excluding the page fields.

VBA
```
Set objPivRange = objPivot.TableRange1
```

The final lines of code in the CreateChartForPivot macro use this range to specify the source data and then specify the appearance of the chart. Notice that, again, the legend is deleted because it duplicates the information on the third axis of a 3-D column chart.

VBA
```
With objChart
    .SetSourceData objPivRange
```

```
    .ChartType = xl3DColumn
    .Legend.Delete
End With
```

The following code sample shows the entire listing for the CreateChartForPivot macro.

VBA

```
Sub CreateChartForPivot()
    ' Creates a chart based on a PivotTable report.
    Dim objPivot As PivotTable, objPivotRange As Range, objChart As Chart

    ' Call the CreatePivot macro to create a new PivotTable report.
    CreatePivot

    ' Determine whether the user deleted the PivotTable report,
    ' and if so, exit the macro.
    If ActiveSheet.PivotTables.Count = 0 Then Exit Sub

    ' Access the new PivotTable from the sheet's PivotTables collection.
    Set objPivot = ActiveSheet.PivotTables(1)

    ' Add a new chart sheet.
    Set objChart = Charts.Add

    ' Create a Range object that contains
    ' all of the PivotTable data, except the page fields.
    Set objPivotRange = objPivot.TableRange1

    ' Specify the PivotTable data as the chart's source data.
    With objChart
        .SetSourceData objPivotRange
        .ChartType = xl3DColumn
        .Legend.Delete
    End With
End Sub
```